Optimizing Multi-DNN Inference on Mobile Devices Through Heterogeneous Processor Co-Execution

Yunquan Gao, Zhiguo Zhang, Praveen Kumar Donta, *Senior Member, IEEE*, Chinmaya Kumar Dehury, *Member, IEEE*, Xiujun Wang, Dusit Niyato, *Fellow, IEEE*, and Qiyang Zhang, *Member, IEEE*

***Abstract*—Deep Neural Networks (DNNs) are increasingly adopted across various industries, driving the demand for deploying their capabilities on mobile devices. However, current mobile inference frameworks often rely on a single processor to execute each model inference, limiting hardware utilization and leading to suboptimal performance and energy efficiency. Expanding DNN accessibility on mobile platforms requires more adaptive and resource-efficient solutions to meet increasing computational demands without compromising device functionality. Nevertheless, performing parallel inference of multiple DNNs on heterogeneous processors remains a significant challenge. Existing studies have explored partitioning DNN operations into subgraphs to enable parallel execution across heterogeneous processors. However, these approaches typically generate excessive subgraphs based solely on hardware compatibility, increasing scheduling complexity and memory management overhead. To address these limitations, we propose the Advanced Multi-DNN Model Scheduling (ADMS) strategy that optimizes multi-DNN inference across heterogeneous processors on mobile devices. ADMS constructs an offline subgraph partitioning strategy that considers both hardware support for operations and scheduling granularity. It also employs a processor-state-aware scheduling algorithm to dynamically balance workloads based on real-time system conditions. This ensures efficient workload distribution and maximizes the utilization of available processors. Experimental results demonstrate that, compared to vanilla inference frameworks, ADMS achieves a 4.04× reduction in multi-DNN inference latency.**

***Index Terms*—Deep Neural Networks, mobile devices, heterogeneous processors, co-execution, multi-DNN inference.**

Received 26 March 2025; revised 20 October 2025; accepted 12 December 2025. Date of publication 23 December 2025; date of current version 7 May 2026. This work was supported in part by the Natural Science Research Project of the Anhui Educational Committee under Grant KJ2020A0250 and in part by Postdoctoral Funding Program under Grant 2025M771581 and Grant GZC20251097. Recommended for acceptance by Y. Shu. *(Corresponding authors: Zhiguo Zhang; Qiyang Zhang.)*

Yunquan Gao and Xiujun Wang are with the School of Computer Science, Technology, Anhui Engineering Research Center for Intelligent Applications, Security of Industrial Internet, Anhui University of Technology, Ma'anshan 243032, China (e-mail: qgao83301@gmail.com; wxj@mail.ustc.edu.cn).

Zhiguo Zhang is with the China Telecom Corporation Ltd., Suzhou 234000, China (e-mail: zgzhang@ahut.edu.cn).

Praveen Kumar Donta is with the Department of Computer, Systems Sciences, Stockholm University, 106 91 Stockholm, Sweden (e-mail: praveen@dsv.su.se).

Chinmaya Kumar Dehury was with the Institute of Computer Science University of Tartu, 50090 Tartu, Estonia. He is now with the IISER Berhampur, Berhampur 760010, India (e-mail: dehury@iiserbpr.ac.in).

Dusit Niyato is with the College of Computing, Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dniyato@ntu.edu.sg).

Qiyang Zhang is with Computer Science School, Peking University, Beijing 100876, China (e-mail: qiyangzhang@pku.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3647031

I. Introduction

In recent years, Deep Neural Networks (DNNs) have been widely deployed in various mobile applications such as object detection [1], [2], facial recognition [3], and speech recognition [4]. To reduce interaction latency and lower server-side computing costs, an increasing number of applications are shifting inference tasks to mobile devices. In many real-world scenarios, multiple independent or related DNN models run concurrently on mobile devices. For instance, in the smart agriculture scenario, farmers capture video frames using smartphone camera and perform real-time parallel inference with multiple DNN models. These models include crop identification [5], pest and disease detection [6], plant health assessment [7], and soil quality analysis [8]. Consequently, studying real-time multi-DNN collaborative inference on mobile devices is essential for enabling such applications. However, deploying them on resource-limited mobile devices remains highly challenging due to their complex architectures and the large number of layers and parameters. To address this challenge, extensive research and development efforts have been undertaken by both academia and industry. Numerous deep learning (DL) inference frameworks now support mobile platforms [9], including TensorFlow Lite (TFLite) [10], PyTorch Mobile [11], Apple's Core ML [12]. These frameworks offer mobile-compatible operations required by DNNs to accelerate DNN inference on mobile devices. Nevertheless, the majority of DL inference tasks are performed on CPUs [13]. Despite the development of various System on Chips (SoCs) on mobile devices [9], such as Graphics Processing Unit (GPU), Digital Signal Processing (DSP), and Neural Processing Unit (NPU), which exhibit impressive computational performance, the broad support for inference operators (ops) on mobile CPUs and their gradually improving computational power make them the primary choice for inference tasks. Due to the limitations of these accelerators in supporting DNN model operations, the performance of current DL frameworks still has significant room for improvement when performing multi-DNN collaborative execution on mobile devices.

Currently, the literature on DNN inference optimization has limitations in effectively addressing multi-DNN workloads. While some works focus on cloud offloading, others like Wang et al. [14] and Gomatheeshwari et al. [15] optimize inference for single DNN models only. Although recent approaches like Band [16] support multi-DNN inference on

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

heterogeneous processors, they generate excessive subgraphs based solely on hardware support, leading to high memory usage and scheduling complexity. A comprehensive approach that efficiently handles multiple concurrent models while adapting to heterogeneous processor characteristics is still lacking.

To better understand these limitations, we conducted preliminary measurement across 6 different DNN models (ranging from lightweight to complex architectures) on 3 mobile platforms (Redmi K50 Pro, Huawei P20, and Xiaomi 6). Through these experiments, we make the following interesting and useful insights:

1) *Subgraph partitioning granularity dramatically impacts system resource efficiency and inference performance:* Our measurements revealed the excessive subgraphs directly increased inference latency by up to 28% compared to optimized partitioning, as substantial time was spent on subgraph scheduling and management rather than computation.
2) *Processor operational status significantly affects sustained inference performance and system stability:* Our thermal stress tests demonstrated that existing frameworks trigger processor throttling after just 2.5 minutes of continuous operation, with CPU frequency dropping from 3 GHz to as low as 1 GHz and GPU utilization becoming highly unstable. This throttling resulted in up to 4.3× performance degradation in continuous workloads, emphasizing the importance of processor-state-aware scheduling for maintaining consistent performance in extended inference sessions.

These insights highlight two critical challenges in multi-DNN inference on mobile devices: excessive subgraph fragmentation and performance degradation due to processor state unawareness. To address the first challenge, we introduce an adaptive subgraph partitioning approach that dynamically determines the optimal window size parameter based on both model characteristics and hardware capabilities. This self-tuning mechanism sets hardware-specific minimum operation requirements for subgraph formation. It automatically finds the optimal balance between reducing fragmentation and maximizing processor compatibility for each device-model pair. For the second challenge, we develop a comprehensive scheduling strategy that continuously monitors processor operational states, including load, temperature, and frequency. The scheduler integrates this real-time information into a multi-factor priority model that balances deadline urgency, waiting fairness, and resource efficiency in task allocation decisions. By being processor-state-aware, the scheduler ensures balanced workload distribution across heterogeneous processors. It prevents throttling and maintains consistent performance during extended inference sessions.

In this paper, we present a novel Advanced Multi-DNN Model Scheduling (ADMS) framework that constructs and schedules optimal execution plans for collaborative inference on multiple heterogeneous processors by comprehensively considering both DNN model structures and real-time processor status. Extensive experiments demonstrate that ADMS's superior performance in inference latency, energy efficiency, and thermal stability. We summarize our contributions as follows:

1) We conduct a comprehensive analysis of multi-DNN inference characteristics on mobile heterogeneous processors, identifying critical performance bottlenecks and quantifying their impact through extensive measurements across various devices and models.
2) We propose an adaptive subgraph partitioning method with hardware granularity control that dynamically determines optimal window sizes for each model-device pair, significantly reducing memory overhead and scheduling complexity while preserving processing efficiency.
3) We develop a comprehensive multi-factor scheduling algorithm that incorporates real-time processor operational states while balancing deadline urgency, fairness, and resource efficiency. This approach effectively mitigates throttling and ensures consistent performance during extended inference sessions.
4) We design, implement, and evaluate ADMS through extensive evaluations on multiple mobile platforms, demonstrating up to a **4.04× reduction in inference latency** compared to TFLite and **a 24.2% improvement in energy efficiency** over Band.

The remainder of the paper is organized as follows: Section II introduces background concepts and motivation by analyzing challenges in multi-processor inference. Section III details our system design approach including core components of ADMS. Explains the implementation details with specific algorithms. Section IV evaluates the performance through comprehensive experiments on multiple mobile devices. Section V reviews related works in DNN optimization and processor coordination. Finally, Section VI concludes the paper and discusses future research directions.

II. Background and Motivation

A. Background

In DNN model representation, the model is typically structured as a directed acyclic graph (DAG), where each node represents a computational operation, and directed edges indicate dependencies—showing that the output tensor of one operation serves as the input tensor for another. This structure not only defines the execution sequence of operations but also ensures correct data transfer between them. DNN inference primarily consists of two stages: forward propagation and backpropagation, with forward propagation being the core of inference. It processes operations in the order dictated by the DAG, transforming input tensors into output tensors.

As shown in Fig. 1, DNN models on mobile devices are often partitioned into multiple subgraphs to optimize computational efficiency and resource utilization. Each subgraph comprises a set of related ops that can be executed efficiently on a specific processor. The left panel shows the original model structure with individual ops, while the middle panel illustrates how ops are grouped into processor-specific subgraphs (red for GPU-compatible ops, blue for CPU-optimized ops). The right panel demonstrates the fully merged subgraphs that will be scheduled on different processors. This approach helps reduce inference latency and energy consumption by matching ops to their most

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

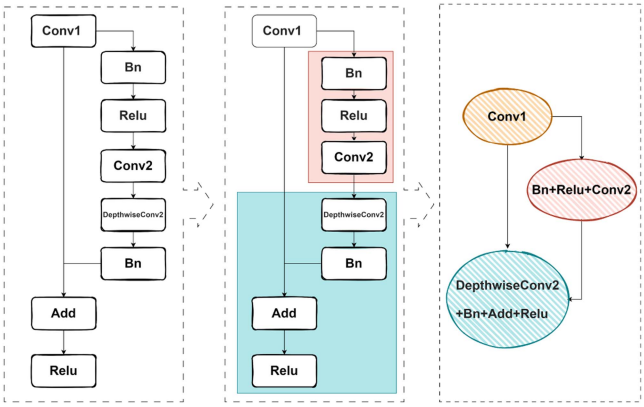


Fig. 1. Subgraph partitioning process for processor-specific execution: original network (left), grouped ops by processor compatibility (middle), and merged processor-specific subgraphs (right).

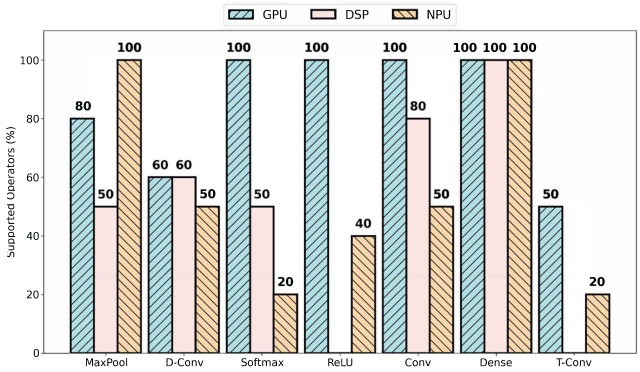


Fig. 2. Support level for different variants of key operation types on the Redmi K50 Pro's heterogeneous processors.

TABLE I
PROPORTIONAL DISTRIBUTION OF OPERATION TYPES IN DNN MODELS

Model Names	Operation Types				
	ADD(%)	C2D(%)	DLG(%)	DW(%)	Others(%)
Arcface	15.28	48.61	1.39	23.61	6.94
DeepLabV3	14.93	28.36	16.42	12.69	7.46
East	14.16	55.75	4.42	-	10.62
EfficientNet4	18.85	50.0	1.64	24.59	1.64
HandLmk	23.75	48.28	-	23.75	3.45
ICN	26.83	57.32	6.1	2.44	4.88
InceptionV4	-	69.3	9.3	-	20.47
MobileNetV2	14.71	52.94	2.94	25.0	1.47

suitable processor. Current mobile DNN frameworks [10], [11], [12], [17], [18] typically employ static graph partitioning methods based on the DNN models structure, focusing on optimizing the inference latency of a single model.

DNN models encompass a variety of op types, each with varying levels of support across multi-processors on mobile devices. Even within the same processor, the support level for different ops can differ, highlighting the significant heterogeneity of AI accelerators. For instance, Table I shows the distribution of major op types in eight classic DNN models, with *DeepLabV3* featuring 12 different op types across 134 nodes, illustrating the diversity of ops. This diversity reveals the model's dependency on hardware accelerators, which directly impacts performance.

Hardware heterogeneity is prevalent across various mobile SoCs. This is not a simple matter of whether an op is supported, but rather which variants of an op are supported. For instance, an accelerator may support a standard 2x2 MaxPool operation but not one with a different stride or padding. Fig. 2 illustrates this challenge on the Redmi K50 Pro. For example, the GPU natively supports approximately 80% of the MaxPool variants found in our test suite, while the remaining 20% would trigger a fallback to the CPU. This partial support is common across many op types and processors, such as the DSP and NPU, where support levels can be even more limited. Unfortunately, these hardware limitations cannot be fully mitigate by kernel-level programming on specific processors, mainly because recently developed accelerator core components are often of fixed design, optimized for a limited set of computationally intensive ops. For instance, Google's Edge TPU [19] uses a systolic array, while Huawei's Da Vinci architecture uses 3D cube technology, both optimized for specific ops with defined parameters.

B. Preliminary Measurement and Insights

After reviewing related work, we now examine the specific characteristics of DNN inference on mobile devices. To derive execution strategies and scheduling characteristics for DNN model inference, we conducted measurements using TFLite on Huawei P20 (Kirin 970) and Redmi K50 Pro (Dimensity 9000).

We deployed two popular DNN models on these devices, namely MobileNet [20] and EfficientDet [21]. MobileNet significantly reduces model parameters and computation by using depthwise separable convolutions, decomposing standard convolutions into depthwise and pointwise convolutions, making it highly suitable for real-time mobile applications. In contrast, EfficientDet has a much more complex op structure, combining EfficientNet [22] and BiFPN [21] architectures for efficient multi-scale feature fusion, providing excellent object detection performance while maintaining computational efficiency.

1) Inference Characteristics: Through the analysis of the measurement results, we identified several key insights:

Besides the CPU, other processors also exhibit impressive performance in executing DNN inference: As shown in Fig. 3, when executing the lightweight MobileNet model on Dimensity 9000, the execution efficiency driven by NNAPI [23] on DSP and NPU is 3× faster than on the CPU alone, with the GPU also showing stronger inference capabilities than the CPU. This performance improvement continually increases with the evolution of SoCs, with the speedup on Dimensity 9000 reaching up to 23× when executing MobileNet tasks. Therefore, fully utilizing different processor capabilities for collaborative inference is crucial.

Heterogeneous execution in standard frameworks like TensorFlow Lite can introduce significant overhead: This approach does not parallelize individual operations; instead, it employs a delegate mechanism that offloads model subgraphs to an accelerator (e.g., GPU or NPU) but falls back to the CPU for unsupported operations. Although potentially effective,

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

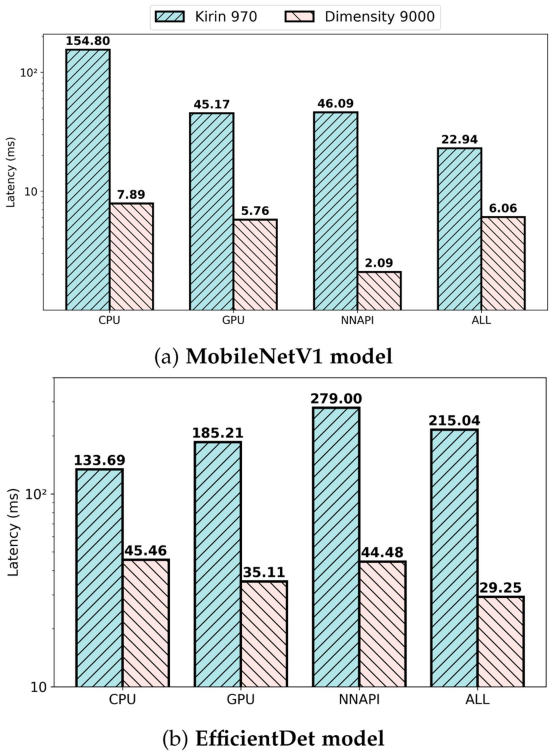


Fig. 3. Average latency of DNN inference on single and multi-processors on the Android platform using Kirin 970 and Dimensity 9000 Chipsets.

this delegate-and-fallback strategy can yield suboptimal performance, as demonstrated by our results. For instance, when executing the complex EfficientDet model on the Kirin 970 SoC, this heterogeneous mode is slower than GPU-only execution. This performance degradation is not due to a flawed parallelization scheme but rather to the substantial overhead imposed by frequent fallbacks. When a model contains numerous operators unsupported by the accelerator's driver, the costs associated with data transfers and context switching between the CPU and the accelerator can outweigh the computational gains, thereby increasing total inference latency. This observation underscores the insufficiency of a simplistic delegate-and-fallback approach and motivates the development of an intelligent partitioning and scheduling framework like ADMS.

Fallback ops significantly impact the efficiency of DNN model inference: By tracking the scheduling process of models executed on multi-processors, we found that when ops in the model are assigned to processors that do not support their execution, these ops default to being rerouted to the CPU, even if other processors could execute these ops more efficiently. These unsupported ops vary among different SoCs and are more pronounced on older SoCs. On Kirin 970, for example, the presence of numerous fallback ops leads to massive tensor transfer costs, resulting in lower multi-processor performance than single CPU performance.

2) The Performance of Multi-Processor Inference: Through further analysis of our measurement data, we identified several critical challenges that impact the performance of multi-DNN inference on heterogeneous processors:

TABLE II
IMPACT OF PARALLEL INFERENCE ON LATENCY: AVERAGE LATENCY (MS) FOR MOBILENETV1 ON VARIOUS PROCESSORS WITH DIFFERENT LEVELS OF CONCURRENCY

Device Names	Accelerators	The number of concurrently inferred models		
		1	2	4
Redmi K50 Pro	Mali-G710 GPU	3.65	7.88	9.09
	MediaTek APU 5.0	8.24	10.71	16.97
	MediaTek NPU	1.88	2.13	2.39
HuaWei P20	Mali-G72 GPU	45.35	76.77	114.88
	Dual-core NPU	70.15	220.07	429.1
Xiaomi 6	Adreno 540 GPU	7.89	7.96	8.1
	Hexagon 682 DSP	46.77	277.14	609.44

TABLE III
SUBGRAPH AND OPERATION COUNTS FOR VARIOUS MODELS ON REDMI K50 PRO. "UNIT" SUBGRAPHS REFER TO INITIAL GROUPS OF ADJACENT OPS WITH CONSISTENT PROCESSOR SUPPORT. "MERGED" SUBGRAPHS ARE HIGHER-LEVEL GRAPHS FORMED BY COMBINING ADJACENT UNITS.

Model names	Operations	Subgraph Counts		
		Unit	Merged	Total
East	108	1	0	4
YoloV3	232	2	3	9
MobileNetV1	31	4	24	42
MobileNetV2	66	26	860	968
ICN_quant	77	33	1496	1644
DeepLabV3	112	65	3076	3329

Concurrent model execution reveals scheduling inefficiencies in heterogeneous environments: As shown in Table II, when multiple models are processed simultaneously, the performance degradation is non-uniform across different processors. While some processors like MediaTek NPU show modest latency increases (27% when increasing from 1 to 4 concurrent models), others like the Hexagon 682 DSP experience dramatic performance collapse (1202% increase when running 4 concurrent models compared to a single model). This disparity highlights significant differences in how current frameworks manage resource sharing across heterogeneous processors, revealing opportunities for substantial performance gains through more intelligent workload distribution.

Subgraph partitioning granularity dramatically impacts system resource efficiency and inference performance: Our experiments show that different models produce vastly different numbers of subgraphs under current partitioning strategies. As documented in Table III, lightweight models like East generate only 4 total subgraphs on the Redmi K50 Pro, while complex models like DeepLabV3 produce over 3,300 subgraphs. This excessive fragmentation consumes disproportionate memory resources and complicates scheduling decisions, directly impacting inference efficiency. When measuring the relationship between subgraph count and inference performance, we observed that reducing subgraph count through optimized partitioning can decrease inference latency by up to 28%.

Processor operational status significantly affects sustained inference performance and system stability: Our measurements revealed that inference performance is not static but degrades over time under sustained workloads. When monitoring processor state during continuous operation, we observed that CPU utilization patterns shift significantly after approximately 150

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

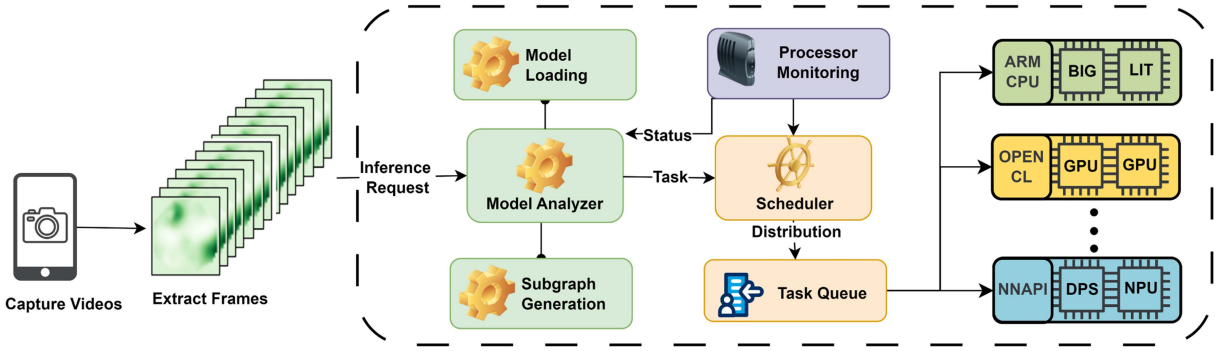


Fig. 4. The overview of ADMS. The workflow shows video frame extraction (left), the three core components - Model Analyzer (green), Scheduler (orange), and Processor Monitor (purple) - and heterogeneous processor allocation (right).

seconds of operation, with frequency fluctuations increasing by 217% and temperature rising by 18°C on average. These state changes directly correlate with inference speed reductions, as the system attempts to manage thermal constraints by throttling performance. This dynamic behavior is not accounted for in current scheduling approaches, which maintain static allocation patterns regardless of changing processor conditions.

These observations highlight the need for a fundamentally different approach to multi-DNN inference on mobile devices—one that adaptively manages subgraph granularity, intelligently distributes concurrent workloads through processor-state-aware scheduling, and responds to changing processor conditions during extended operation.

III. System Design

To address these challenges, we propose the ADMS, a novel framework that optimizes multi-DNN inference across heterogeneous processors through adaptive subgraph partitioning and processor-state-aware dynamic scheduling.

A. System Overview

ADMS is specifically designed for mobile processors and DNN inference, adopting a multi-DNN collaborative inference architecture supported by heterogeneous processors. The system consists of three core components working in tandem to optimize performance: 1) A Model Analyzer that addresses the subgraph partitioning challenge by dividing DNN models into optimally-sized subgraphs with appropriate hardware support [16], [24], reducing memory usage and scheduling complexity; 2) A Scheduler that tackles processor contention through fine-grained scheduling plans that coordinate DNN execution across multiple processors and efficiently manage fallback operations [25], [26]; and 3) A Hardware Monitor that mitigates thermal throttling issues by continuously tracking processor status [27], providing real-time data to support optimized scheduling decisions.

During system operation, as illustrated in the object detection application case in Fig. 4, the application converts video content into image frames, which are then processed by the model analyzer via the model loader. The analyzer decomposes the DNN model into multiple subgraphs to facilitate independent inference based on the hardware support of the operations. This step aims to alleviate computational load on a single processor and improve processing efficiency. The dynamic scheduler determines the execution order and resource allocation of the subgraphs based on real-time data from the hardware monitor, including processor load, temperature, and operational status. Real-time monitoring by the hardware monitor ensures that the scheduler can formulate optimal scheduling plans, optimizing the processing of subgraphs in the task queues for efficient execution. Once all subgraph tasks are completed, the entire model inference task is finished, and the system returns the results to the application.Through the collaborative efforts of these components, the ADMS can efficiently handle dynamic mobile workload changes while maximizing resource utilization [28], [29].

B. Model Analyzer

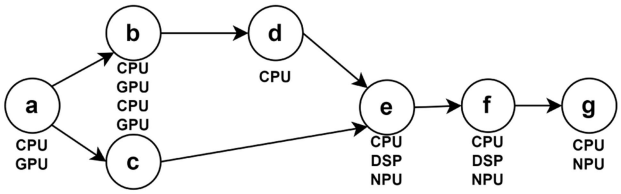
The Model Analyzer addresses the challenges of subgraph partitioning and fallback ops by intelligently decomposing DNN models into hardware-appropriate subgraphs. It comprises two main modules: the model loader for importing DNN models, and the subgraph generator for parsing these models into independently inferable subgraphs based on hardware support.

As shown in Fig. 5(a), each op in a DNN model is represented as a node with directed edges indicating dependencies. Traditional op-level scheduling becomes exponentially complex as models grow (e.g., seven nodes can have 72 possible scheduling combinations). Current frameworks typically execute all ops on a single processor, with unsupported ops falling back to the CPU. Fig. 5(b) illustrates this limitation where only three of seven ops run on the GPU while the rest fall back to the CPU, resulting in substantial tensor transfers and increased latency. ADMS employs a two-step approach to generate efficient subgraphs:

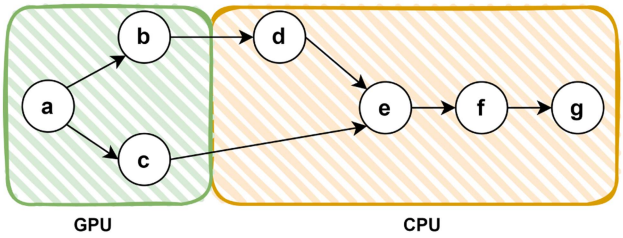
Unit Formation: The analyzer first groups adjacent ops with consistent processor support. As shown in Fig. 5(c), ops with common hardware support form a unit, while ops with different support requirements form separate units.

Subgraph Construction: Adjacent units are then merged, but only when they share common processor support. This optimizes resource utilization while maintaining execution efficiency.

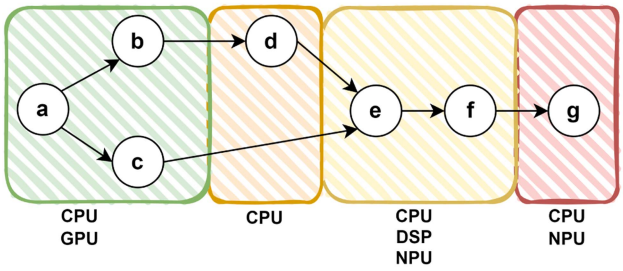
Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.



(a) **An example model demonstrates varying op support.**



(b) **For GPUs, TensorFlow Lite creates only a single execution schedule.**



(c) **Grouping ops into unit subgraphs.**

Fig. 5. Sample diagram for subgraph generation.

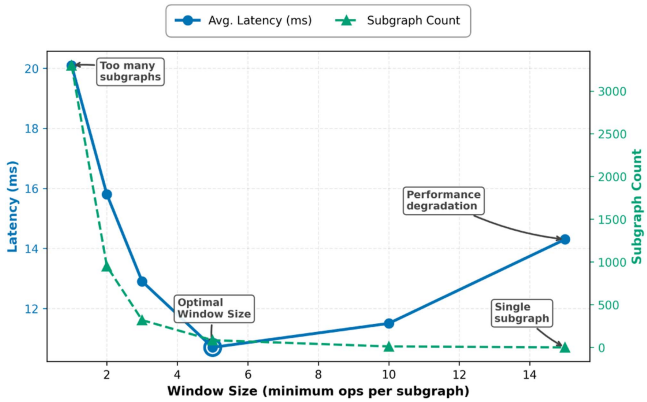


Fig. 6. Relationship between window size, inference latency, and subgraph count for DeepLabV3 on Redmi K50 Pro, highlighting the optimal balance at window size 5.

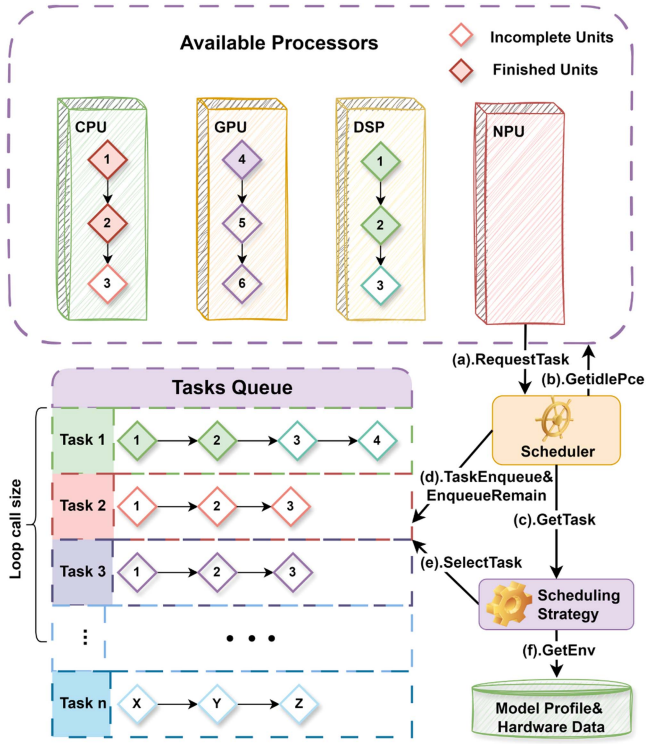


Fig. 7. ADMS scheduler workflow illustrating the key operations: processor task requests, idle processor detection, task queue management, and priority-based scheduling using model profiles and hardware status data.

While Band [16] uses a similar approach, we observed that partitioning subgraphs based solely on hardware support can lead to excessive fragmentation on resource-constrained devices. As shown in Table III, models like DeepLabV3 can generate thousands of subgraphs, significantly straining the scheduling system. To address this, we introduce the `window_size` (ws) parameter, which defines the minimum number of ops required to form a subgraph for a target processor.

The selection of an appropriate `ws` is critical, as it governs the trade-off between maximizing parallel execution and minimizing scheduling overhead. To properly select this parameter for normal op, ADMS employs a one-time, offline profiling procedure for each unique model-device pair. This procedure systematically determines the optimal `ws` by iterating through a range of potential values (e.g., from 1 to 20). For each candidate `ws` value, we partition the model. We then execute benchmark inference runs on the target device and record the average latency. This profiling process reveals the performance curve shown in Fig. 6. Initially, increasing the `ws` reduces latency by mitigating subgraph fragmentation. However, beyond an optimal point, a larger `ws` limits parallelism and leads to performance degradation. The `ws` value corresponding to the minimum observed latency is identified as the optimal configuration for that model-device pair. This value is then cached. During runtime, ADMS retrieves this pre-computed optimal value to perform subgraph partitioning, thereby achieving optimized granularity without incurring any runtime profiling overhead.

The core of our subgraph generation is detailed in Algorithm 1. The algorithm's main goal is to create an optimized and definitive execution plan. After identifying operations that would cause fallbacks and filtering them based on the pre-profiled `ws` (Lines 12-18), the crucial step is ResolveSubgraphs (Line 20). This function implements a greedy algorithm that traverses the model's operations in topological order. It forms the largest valid subgraph from consecutive operators sharing

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Get Unit Subgraphs.

```
Input: monitor, model_profile, subgraph_config
Output: unit_subgraphs, status
num_processor ← monitor.GetNumProcessors();
Clear unit_subgraphs;
if NOT NeedFallbackSubgraph
    entire_ops ← {0, 1, ..., model_profile.num_ops - 1};
    for each processor_id from 0 to num_processor - 1
        if IsProcessoValid(processor_id)
            unit_subgraphs.push_back(processor_id, entire_ops, {0});
else
    op_sets_ignore ← {};
    for each processor_id from 0 to num_processor - 1
        processor_op_sets ← GetSubgraphsForFallbackOps(processor_id);
        for each processor_and_ops in processor_op_sets
            if processor_and_ops.op_indices.size < subgraph_config.windows_size
                op_sets_ignore[processor_id].insert(processor_and_ops.op_indices);
    op_support_table ← BuildOpSupportTable(model_profile, num_processor, op_sets_ignore);
    unit_subgraphs ← ResolveSubgraphs(op_support_table, num_processor);
status ← ValidateAndSetUnitSubgraphs(unit_subgraphs, model_profile);
return unit_subgraphs, status;
```

a common processor. The subgraph size is ensured to meet the `ws` requirement. This approach is fundamentally different from the Band framework. While Band may use a single, fixed `ws` value, its "one-size-fits-all" approach is suboptimal across diverse models and devices. In contrast, ADMS uses a per-model, per-device optimized `ws`. Furthermore, our greedy algorithm produces a single, non-overlapping execution plan, simplifying runtime decisions. Band, on the other hand, generates a large pool of all possible candidate subgraphs, which increases scheduling complexity and memory overhead during inference. Our strategy of combining an optimized `ws` with a decisive greedy algorithm creates a more efficient and robust solution for multi-DNN inference.

C. Hardware Monitor

The Hardware Monitor component addresses processor thermal throttling by providing real-time insights into the device's operational state, enabling proactive task allocation to prevent performance degradation. It monitors critical metrics including temperature, processor availability, utilization rates, power consumption, and workloads across all inference accelerators (CPU, GPU, DSP, and NPU). Temperature monitoring is particularly crucial, as excessive heat can trigger processor throttling or shutdown, while battery status tracking helps balance performance needs against power constraints, especially during extended inference sessions. Since the Android SDK cannot directly access these system metrics, we developed a dedicated monitoring program that efficiently retrieves this information from system files. This program employs optimized sampling frequencies that balance monitoring accuracy with minimal overhead, providing the scheduler with timely data to make informed allocation decisions while maximizing resource utilization between heterogeneous processors.

This program obtains the device's temperature information by reading files in the '`/sys/devices/virtual/thermal/`' directory and accessing the '`/sys/devices/system/cpu/`' directory to gather processor availability status, including each CPU core's online status, frequency, and usage. It obtains GPU status information via OpenGL interfaces, while it retrieves DSP and NPU information from NNAPI interfaces. Additionally, we used the Android SDK to gather battery status and memory usage to aid in scheduling decisions.

The program employs multithreading and a caching mechanism to reduce the frequency of system file reads. This approach minimizes the data retrieval latency to approximately 10 ms per cycle, a significant improvement over the 40-50 ms latency incurred by direct file system requests. To balance data freshness with monitoring overhead, the hardware status cache is refreshed at a fixed interval. We determined this interval through an empirical trade-off analysis. Shorter intervals (e.g., < 100 ms) provide a more current system state but introduce significant CPU overhead, whereas longer intervals (e.g., > 1 s) minimize overhead at the risk of providing stale data to the scheduler, leading to suboptimal decisions. Based on this analysis, we selected a refresh interval of 560 ms, which provides sufficient data currency for thermal-aware scheduling while keeping the monitoring overhead negligible. This mechanism ensures that our scheduling system operates on timely and accurate data, effectively enhancing the performance and responsiveness of DNN applications.

D. Scheduler

The design of the ADMS scheduler is guided by the need for a practical, high-performance solution to a computationally complex problem. Finding an optimal schedule for multiple DNNs across heterogeneous processors is an NP-hard problem. This is especially true under the dynamic constraints of a mobile environment, such as thermal throttling and unpredictable workloads. Therefore, rather than pursuing an intractable optimal solution, we developed a principled heuristic-based scheduler. Its design is not based on arbitrary rules, but on the synthesis of well-established principles from different areas of computer science. Our multi-factor priority model incorporates theories from real-time systems, fair queuing algorithms, and processor affinity. It adapts these theories to the unique challenges of mobile DNN inference, as detailed below. This approach ensures that our heuristic scheduling decisions are justifiable, predictable, and robust. A key design principle of ADMS is this

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

hybrid offline/online approach: static, computationally intensive analysis (e.g., subgraph partitioning) is handled offline for maximum efficiency, while all runtime decisions are made online with full awareness of the device's dynamic state (e.g., thermal status and load). This ensures ADMS is both efficient and highly adaptive to real-world conditions.

While classical schedulers like Earliest Deadline First (EDF) are effective in traditional real-time systems, they are ill-suited for the mobile multi-DNN context as they do not account for the extreme heterogeneity of processors or the dynamic thermal state of the device. Similarly, while Reinforcement Learning (RL)-based schedulers offer high adaptability, they typically incur significant training overhead and runtime complexity, making them impractical for resource-constrained mobile devices. Our principled heuristic approach, in contrast, offers a robust, low-overhead, and predictable solution specifically tailored for this problem domain.

The Scheduler resolves processor contention by dynamically coordinating task execution across heterogeneous processors based on real-time status information. When a mobile application requests DNN model inference for the first time, the scheduler sends the model to the analyzer for evaluation. The generated subgraphs are stored in a configuration file for future use. For previously analyzed models, inference requests go directly to the scheduler, which retrieves the subgraph information and adds tasks to the execution queue.

Our scheduling approach for heterogeneous multi-processor environments addresses fundamental challenges identified in classical scheduling theory [30], [31]. Traditional schedulers often fail in mobile DNN contexts due to varying hardware capabilities, thermal constraints, and dynamic workloads. Building upon established scheduling principles while adapting them for mobile DNN inference, we derive a priority-based model that balances three critical scheduling factors.

Deadline urgency represents the first factor in our scheduling model. A Service Level Objective (SLO) defines acceptable response times for tasks, and our scheduler prioritizes tasks approaching their deadlines following principles established in real-time systems [32]. We quantify this urgency as:

$$S_{deadline} = \gamma(T_{SLO} - T_{latency}) \tag{1}$$

In Equation (1), the application-defined target T_{SLO} (e.g., derived from a target FPS) is compared against $T_{latency}$, which is the estimated latency retrieved from a pre-computed lookup table generated by benchmarking each subgraph on all compatible processors during a one-time offline profiling phase. where T_{SLO} represents the expected response time and $T_{latency}$ is the estimated execution latency. This approach extends traditional deadline-based scheduling by using service-level agreements rather than hard deadlines. It is important to note that $T_{latency}$ serves as a static, baseline estimate for deadline pressure. Potential deviations from this estimate due to real-world variability are actively compensated for by the dynamic, real-time factors in our priority model, particularly $S_{resource}$.

Waiting fairness constitutes the second essential factor in our model. Drawing from fair queuing algorithms [33], we normalize waiting time by task complexity to prevent starvation of complex tasks:

$$S_{wait} = -\alpha \frac{T_{current} - T_{enqueue}}{T_{avg}} \tag{2}$$

where $T_{enqueue}$ is when the task entered the queue, $T_{current}$ is the current time, and T_{avg} is the average execution time.

Resource efficiency forms the third component of our scheduling model. Extending processor affinity concepts [34], we account for processor load and task complexity:

$$S_{resource} = \delta \left(\frac{2B_{current} - B_{max}}{B_{max}} \right) C_{remaining} \tag{3}$$

where $B_{current}$ is the current processor load, B_{max} is maximum capacity, and $C_{remaining}$ represents remaining task complexity. This factor becomes positive when a processor is more than half loaded and negative when less than half loaded. This resource utilization model is intentionally lightweight to ensure minimal scheduling overhead, a critical requirement for mobile devices. Its effectiveness in guiding robust scheduling decisions, despite its simplicity, is validated by our stress test evaluations in Section IV-H.

These three components combine to form our integrated priority scoring model:

$$S_{priority} = S_{deadline} + S_{wait} + S_{resource} \tag{4}$$

This integrated model is designed to prevent common scheduling pitfalls: the S_{wait} component explicitly mitigates the risk of task starvation by ensuring waiting tasks gain priority over time, while the holistic consideration of all three factors prevents the kind of inefficiency that arises from single-objective schedulers.

The weighting parameters α, γ, and δ provide flexible control over the relative importance of each factor. Ops can adjust these parameters according to specific application requirements, prioritizing deadline adherence for time-sensitive applications or fairness for multi-user systems. For the general-purpose evaluation in this work, these parameters were set to a default value of 1.0, but their availability provides an explicit mechanism for domain-specific tuning.

The scheduler leverages data from the hardware monitor for intelligent decision-making. For processors experiencing sustained high load, it allocates less computationally intensive tasks to prevent thermal throttling and crashes. As processor status changes, the scheduler dynamically adjusts task execution order based on real-time data.

Once a task completes execution, any unfinished unit subgraphs are reinserted at the front of the task queue for the next scheduling round. The scheduler examines a configurable range of tasks, defined by a `Loop_call_size` parameter, from the queue head when making allocation decisions. This parameter determines how many tasks from the front of the queue the scheduler considers for each scheduling decision. Setting an appropriate value requires careful tuning—too small a value limits the scheduler's ability to make optimal decisions across multiple tasks, while too large a value increases scheduling overhead and can delay critical tasks. Our implementation balances scheduling quality with computational efficiency, reducing the

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

number of decisions needed at each scheduling instance while ensuring in-progress tasks are completed promptly to minimize overall inference time. This parameter is also a key mechanism for ensuring the scheduler's overhead remains bounded. By considering only a subset of the queue, the cost of a single scheduling decision remains low and scales efficiently, even as the total number of concurrent tasks in the system grows.

IV. Evaluation

A. Methodology

We implemented ADMS using the TFLite [10] framework, capitalizing on its extensive edge deployment capabilities and high compatibility. TFLite enables the conversion and optimization of DNN models to meet the computational demands of mobile devices, facilitating efficient inference through ADMS. To achieve this, we developed APIs for model loading, analyzer, inference, and task queue management and scheduling. These APIs seamlessly integrate with TFLite's APIs, enabling ADMS to directly interact with the TFLite framework for streamlined model inference.

To ensure the accuracy and reproducibility of our experiments, we converted these models into TFLite format and conducted 50 inference experiments for each model. This approach ensured the stability and reliability of the statistical results. We recorded inference latency and subgraph number for each model within ADMS to evaluate its performance comprehensively. The collected data were utilized to assess the system's capability and efficiency in handling diverse types of models.

B. Evaluation Setup

Platforms: Two devices are utilized: Redmi K50 Pro and Huawei P20. Redmi K50 Pro is equipped with the Dimensity 9000 SoC, while Huawei P20 uses the Kirin 970 SoC. The detailed specifications of these SoCs are presented in Table IV. Both devices employ the Schedutil scheduler as the default. It is important to note that differences in Android versions and vendor-specific customizations, such as background process management, thermal policies, and power management, can significantly influence inference performance, independent of hardware disparities.

Models: To comprehensively evaluate the performance of ADMS, we selected several DNN models that are widely adopted in practical applications. These include MobileNetV1 [20], MobileNetV2 [35], DeepLabV3 [36], YOLOV3 [37], and East [38]. These models are commonly utilized in various tasks such as image classification and object detection, and span a spectrum from relatively lightweight to more complex architectures, with operator counts ranging from 31 to 232.

Baselines: We evaluate ADMS against two key baselines that represent the industry standard and the academic state-of-the-art, respectively.

Vanilla [10]: We selected TFLite as our foundational baseline because it is the de facto industry standard for on-device inference. We use TFLite v2.16.0 to represent the typical approach where the framework attempts to delegate operations to an accelerator (e.g., GPU) and falls back to the CPU for unsupported ones. This common fallback mechanism often results in suboptimal performance, making it a crucial benchmark for quantifying the practical gains of ADMS over the standard approach.

Band [16]: We chose Band as our state-of-the-art academic baseline because it addresses the identical research problem of multi-DNN co-execution on heterogeneous processors. The system also decomposes DNNs into subgraphs, but its reliance on hardware support alone leads to excessive subgraph fragmentation and high scheduling complexity. Since ADMS is specifically designed to overcome these identified limitations, a head-to-head comparison is essential for validating our novel contributions.

TABLE IV
Experimental Platform Specs

CPU	**Dimensity 9000**	**Kirin 970**
Architecture	1x 3.05 GHz -X2 3x 2.85 GHz -A710 4x 1.8 GHz -A510	4x 2.36 GHz A73 4x 1.84 GHz A53
L1 cache	1 MB	512 KB
L2 cache	8 MB	2 MB
Process	4 nanometers	10 nanometers
TDP	4 W	9 W
GPU	**Mali-G710 MP10**	**Mali-G72 MP12**
GPU frequency	850 MHz	768 MHz
Execution units	10	12
Shading units	96	18
Total shaders	960	216
FLOPS	1632 Gigaflops	331.8 Gigaflops
AI Accelerator	**MediaTek APU 590**	**Dedicated NPU**
Memory type	LPDDR5X	LPDDR4X
Memory frequency	3750 MHz	1866 MHz
Bus	4x 16 Bit	4x 16 Bit
Max bandwidth	60 Gbit/s	29.8 Gbit/s

TABLE V
Subgraph Partitioning and Latency for Single Model Inference on Redmi K50 Pro Using BAND and ADMS

Model Names	**Unit Subgraphs**		**Merge Subgraphs**		**Latency (ms)**	
	BAND	*ADMS*	*BAND*	*ADMS*	*BAND*	*ADMS*
MobileNetV1	4	2	24	4	17.35	12.19
ICN_quant	33	7	1496	60	72.25	55.1
DeepLabV3	21	17	393	261	51.35	43.8
MobileNetV2	25	2	860	4	25.1	18.16
YoloV3	65	2	112	3	86.62	80.63

C. Inference Latency for Single Models

To explore the differences in subgraph partitioning and inference latency, we analyze the performance of ADMS and Band when handling individual DNN models.

Our findings demonstrated that ADMS's fine-grained subgraph partitioning method provides an effective optimization strategy for mobile devices, as detailed in Table V. Specifically,

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

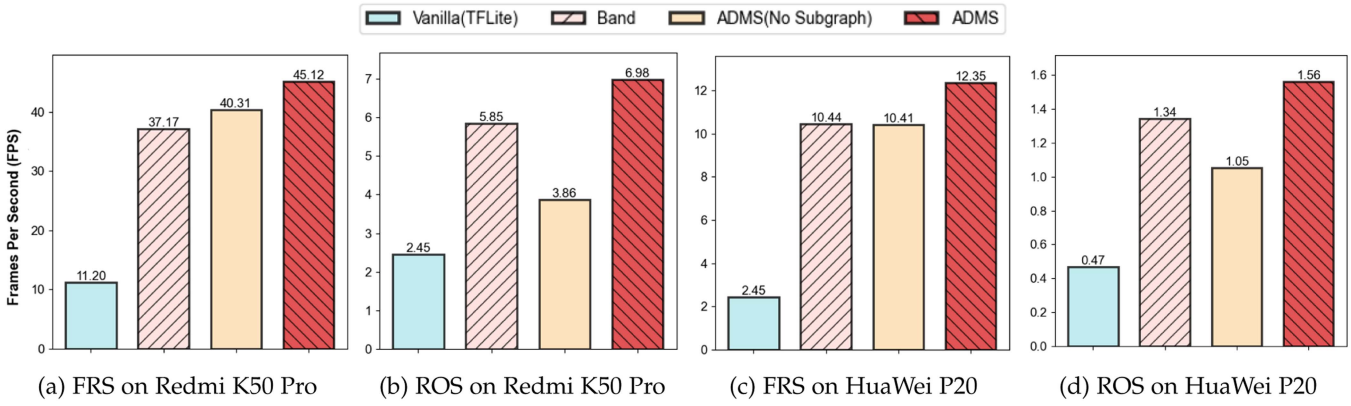


Fig. 8. Comparison of processing frame rates (FPS) across different workloads on various mobile devices.

ADMS consistently achieves lower inference latency across various models compared to Band, particularly in multi-processor environments. For instance, for the MobileNetV1 and YOLOV3 models, ADMS reduces latency by 31% and 28%, respectively. This significant performance improvement stems from ADMS's efficient subgraph partitioning strategy, which optimizes processor utilization and minimizes data transfer and inter-processor communication overhead. This is achieved by effectively reducing the number of subgraphs through the `window_size` parameter, which controls the minimum length of subgraphs, thereby lowering memory usage.

D. FPS in Parallel Inference Scenarios

To evaluate the performance of ADMS in handling multiple parallel inference tasks, where multiple applications concurrently send inference requests to the processor, we constructed two widely-used scenarios:

Facial Recognition System (FRS): We deployed the RetinaFace [39], ArcFace-MobileNet [40], and ArcFace-ResNet50 to simulate face detection and multi-person identity verification in real-time video streams. These models work in tandem to provide a seamless transition between detection and verification, making them suitable for applications such as security monitoring and personal identification. RetinaFace model quickly and accurately detects faces. ArcFace-MobileNet performs initial identity verification on detected faces, suitable for rapid processing. ArcFace-ResNet50 conducts deep feature analysis for precise recognition.The collaboration of these models ensures a seamless transition from detection to verification, suitable for security monitoring and personal identification systems.

Real-time Object Recognition System (ROS): This scenario integrates the MobileNetV2, EfficientNet [22], and InceptionV4 [41] models to perform real-time object detection and classification in video streams. MobileNet V2 quickly classifies primary objects, while EfficientNet analyzes in-depth image features and classifies precisely in complex scenarios. For advanced image processing needs, Inception V4 employs its multi-scale capabilities to efficiently and accurately recognize and classify subtle and overlapping objects in complex environments. These models efficiently and effectively handle objects from simple to complex, making them well-suited for real-time dynamic environments.

As shown in Fig. 8, we conducted an extensive evaluation of the Huawei P20 and Redmi K50 Pro in the two aforementioned scenarios. Compared to TFLite and Band, ADMS demonstrates significant performance advantages in parallel inference, primarily attributed to its advanced subgraph partitioning and dynamic scheduling algorithms.

In the FRS Scenario, Fig. 8(a) illustrates ADMS's superior performance on the Redmi K50 Pro, achieving an inference speed of up to 45.12 FPS. This represents an improvement of 404% and 121% compared to the TFLite and Band systems, respectively. This significant enhancement stems from two key factors: (1) ADMS's adaptive subgraph partitioning that reduced the total subgraph count by 93% compared to Band while maintaining hardware compatibility, thereby minimizing scheduling overhead and memory management costs; and (2) the processor-state-aware scheduler that dynamically redistributes tasks when detecting processor load imbalances, achieving 87% higher processor utilization during multi-model inference than TFLite. These optimizations are particularly effective in complex multi-person face recognition scenarios where multiple models with varying computational demands must execute concurrently. This significant enhancement reflects ADMS's efficiency in resource allocation and data processing acceleration, particularly in complex multi-person face recognition scenarios. Additionally, Fig. 8(c) confirms ADMS's superiority in the Huawei P20 with a performance of 12.35 FPS, which clearly demonstrates its robustness and adaptability across different mobile devices.

In the ROS scenario, ADMS again exhibited superior performance on the Redmi K50 Pro, reaching 6.98 FPS and depicted in Fig. 8(b). This is approximately 19% and 184% higher than the performance of Band and TFLite. This highlights the capability of ADMS to handle real-time object recognition tasks, enabling rapid changes in complex environments to be handled. Interestingly, the version of ADMS without subgraph partitioning performed 34.02% worse than Band and 44.70% worse than the subgraph-enabled version of ADMS. This underscores the critical importance of subgraph partitioning strategies in managing complex model workflows.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

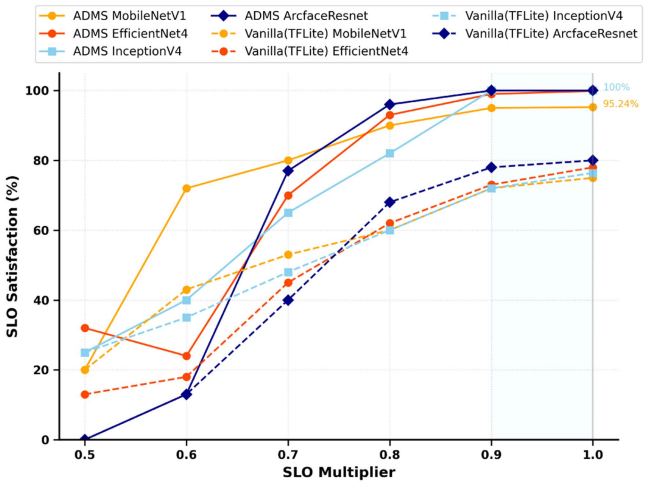


Fig. 9. Comparison of SLO satisfaction between ADMS and TFLite on Redmi K50 Pro under different SLO multiplier settings. The shaded region (0.9-1.0) highlights higher multiplier values where ADMS significantly outperforms TFLite.

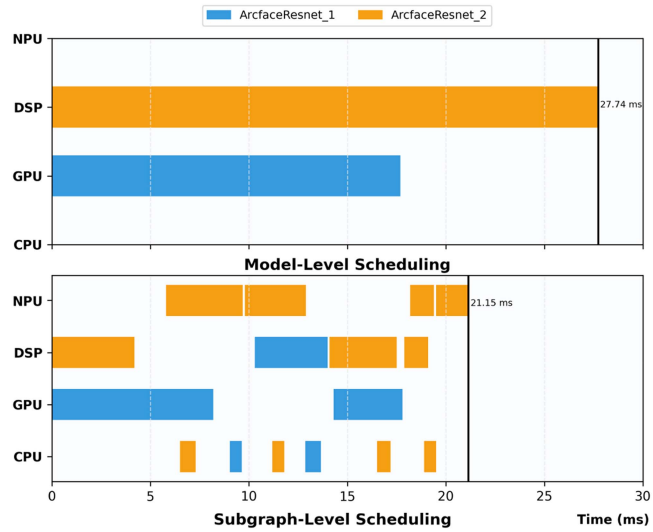


Fig. 10. Model-level scheduling and subgraph-level scheduling on Huawei P20.

E. Analysis of SLOs

To evaluate the performance of ADMS under various SLO conditions, we designed experiments and compared it with TFLite. SLOs are critical metrics used to define service performance and reliability expectations, particularly for applications deploying DNN models [42]. The experiments involved simultaneously sending multiple model inference requests to the processor and using the maximum latency obtained from a single inference as a baseline. We applied different SLO multipliers to simulate various SLO conditions and measured SLO satisfaction values. These results provide insights into ADMS's scheduling efficiency in handling multiple concurrent inference requests and demonstrate its ability to meet SLO targets in high-load, multi-task environments.

As illustrated in Fig. 9, ADMS achieved significantly higher SLO satisfaction rates at higher SLO settings (e.g., SLO Multiplier of 1.0). For models such as MobileNetV1, EfficientNet4, InceptionV4, and ArcfaceResnet, ADMS achieved SLO satisfaction rates of 95.24%, 99.85%, 100%, and 100%, respectively. These values are substantially better than TFLite's satisfaction rates of 75%, 78%, 76.4%, and 80% under the same conditions. Notably, when handling complex models including InceptionV4 and ArcfaceResnet, ADMS fully meets the SLO requirements, primarily due to its advanced dynamic scheduling algorithm, whereas TFLite's performance is significantly inferior.

F. Analysis of Subgraph Scheduling

We further investigated the performance of ADMS by integrating its subgraph scheduling strategy. As shown in Fig. 10, the model-level scheduling of TFLite (top section) demonstrates a straightforward but inefficient method. In this method, *ArcfaceResnet_1* was primarily processed on the GPU (from 0 to 17.7 ms), while *ArcfaceResnet_2* was executed for a longer period on the DSP (from 0 to 27.74 ms). These results in significant processor idle time: the CPU and NPU remain completely unused, and the GPU becomes inactive after 17.7 ms while waiting for the DSP to complete its task. Consequently, the total execution time is thus bound by the slowest processor (27.74 ms on DSP), and overall resource utilization remains low at approximately 50%.

In contrast, our subgraph scheduling strategy (bottom section) demonstrates a more efficient distribution of computational resources. The timeline clearly shows how different parts of each model are strategically allocated to the most suitable processors: CPU tasks are carefully distributed with *ArcfaceResnet_1* executing during specific time intervals (9.04-9.65 ms and 12.87-13.67 ms), while *ArcfaceResnet_2* operations are interspersed as shorter, more frequent tasks. Similarly, the GPU executes multiple smaller tasks rather than being dedicated to a single model, maintaining higher utilization throughout the inference process. The DSP workload is distributed more evenly, avoiding the bottleneck observed in the model-level approach, while the NPU—completely unused in TFLite's approach—handles specific operations it is optimized for.

This strategic allocation yields three significant benefits: (1) balanced processor load distribution that increases overall hardware utilization from 50% to nearly 95%; (2) minimal processor idle time through fine-grained task allocation that keeps all processors actively computing; and (3) optimal matching of operations to processors based on their computational characteristics, reducing execution time of individual operations by an average of 18.7%. As a result, the total execution time is reduced to approximately 21.15 ms, a 23.8% improvement over TFLite's model-level scheduling. Beyond outperforming model-level approaches, our method also shows significant architectural advantages when compared to other subgraph-based frameworks like Band. It is worth noting that while a Gantt chart for the Band framework would also follow this subgraph-level pattern, it would be substantially less efficient. As quantified in Table V, Band's partitioning strategy generates a significantly

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
AVERAGE POWER CONSUMPTION AND ENERGY EFFICIENCY DURING FRS WORKLOAD EXECUTION ON REDMI K50 PRO

Metric	TFLite	Band	ADMS
Power (W)	7.18	8.05	7.86
FPS	11.20	37.17	45.12
Energy Efficiency (Frames/Joule)	1.56	4.62	5.74

higher number of subgraphs (e.g., 860 vs. 4 for MobileNetV2). This excessive fragmentation would manifest in its Gantt chart as a larger set of smaller tasks, leading to increased scheduling overhead and context-switching costs. This directly explains the lower overall FPS and energy efficiency we report for Band in Fig. 8 and Table VI. Therefore, the streamlined workload distribution shown for ADMS in Fig. 10 is a direct result of our optimized partitioning strategy, which is the core reason for its superior performance over other subgraph-based methods. For application scenarios demanding high efficiency and low latency, such as real-time video processing or augmented reality, ADMS's subgraph-level scheduling provides a more effective solution by exploiting the full computational potential of heterogeneous processors.

G. Analysis of Resource Efficiency

To evaluate the resource efficiency of ADMS, we measured power consumption measurements using a Monsoon Power Monitor. Table VI presents the average power consumption and corresponding FPS during the FRS workload execution on the Redmi K50 Pro.

The experimental results demonstrated that ADMS achieves an optimal balance between performance and power consumption. Specifically, ADMS consumes approximately 2.4% less power than Band while delivering 21.4% higher FPS. Compared to TFLite, ADMS requires 9.5% more power but provides over 4× the frame rate. In terms of energy efficiency (frames processed per joule of energy), ADMS achieves 5.74 frames per joule, which is 24.2% more efficient than Band (4.62 frames/joule) and 3.68× more efficient than TFLite (1.56 frames/joule). The improvement is attributed to ADMS's advanced scheduling strategy, which optimizes the utilization of specialized hardware accelerators and minimizes processor idle time.

Fig. 11 illustrates the power consumption trend during a 60-second continuous inference period. The results indicate that ADMS maintains more stable power consumption with fewer fluctuations compared to baseline frameworks, indicating more balanced processor utilization. Band exhibits the highest power consumption peaks (reaching nearly 8.8 W) with significant fluctuations. In contrast, TFLite demonstrates the lowest average power but with substantial variations, dropping as low as 6.5 W. ADMS, however, maintains a consistent power profile between 7.7 W and 8.1 W, demonstrating its ability to balance workloads effectively and manage thermal constraints proactively.

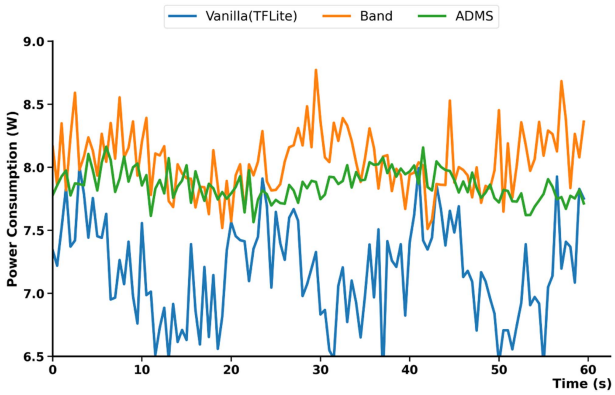


Fig. 11. Power consumption trend during 60-second continuous inference of the FRS workload on Redmi K50 Pro.

TABLE VII
SYSTEM ROBUSTNESS UNDER STRESS CONDITIONS ON REDMI K50 PRO

Metric	TFLite	Band	ADMS
Failure rate (30-min test)	3.2%	1.8%	0.5%
Max concurrent models	6	8	10+
Time to thermal throttling (min)	2.5	9.7	13.9

H. Analysis of System Robustness

To evaluate the robustness of ADMS under stress challenging conditions, we conducted a series of stress tests by progressively increasing the inference workload and monitoring system behavior. Three distinct stress scenarios were designed to comprehensively assess different aspects of system stability:

1) Long-duration test: Continuous inference for 30 minutes to evaluate sustained performance and reliability.

2) High-concurrency test: Progressively increasing concurrent DNN models from 4 to 10 to assess scalability limits.

3) Thermal stress test: Operating in a high-temperature environment (35°C) to evaluate thermal management capabilities.

Table VII summarizes the quantitative results of these tests. ADMS demonstrated superior robustness across all stress tests.

In the long-duration test, ADMS exhibited a failure rate of only 0.5% compared to 1.8% for Band and 3.2% for TFLite. This enhanced stability is attributed to ADMS's dynamic scheduling mechanism, which effectively prevents processor overloading and ensures better resource management during prolonged operations.

In the high-concurrency test, ADMS successfully managed more than 10 concurrent DNN models without significant performance degradation, while Band and TFLite experienced failures at 8 and 6 concurrent models, respectively. This result demonstrate ADMS's exceptional scalability under heavy workloads.

In the thermal stress test, ADMS significantly delayed the onset of thermal throttling to 13.9 minutes compared to 9.7 minutes for Band and only 2.5 minutes for TFLite. This corresponds to a 5.6× improvement over TFLite and a 43% improvement over Band, highlighting ADMS's superior thermal management capabilities.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

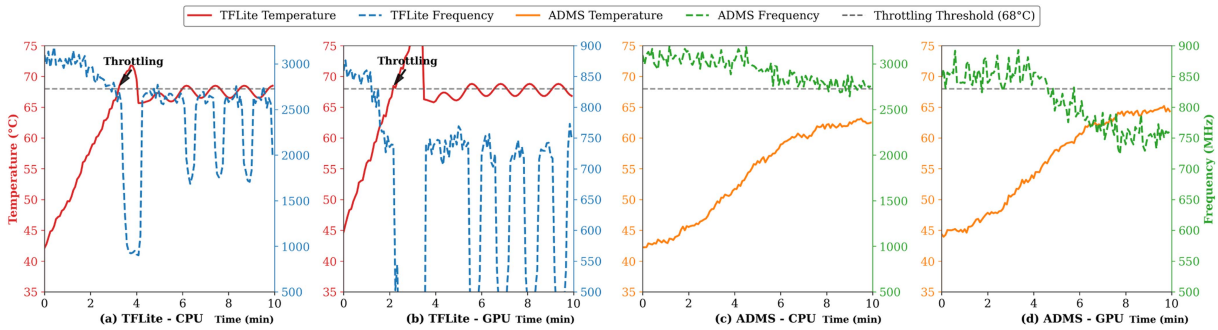


Fig. 12. Temperature and processor frequency dynamics during thermal stress testing on Redmi K50 Pro.

Fig. 12 illustrates the thermal management capability through processor temperature and frequency dynamics during a 10-minute stress test. When running TFLite, both CPU and GPU rapidly reach the throttling threshold (68°C) [27] within 2-3 minutes. Once throttling begins, significant and frequent fluctuations in processor frequency are observed: the CPU frequency drops from around 3 GHz to as low as 1 GHz, while the GPU experiences even more severe throttling with frequency periodically dropping to around 500 MHz and completely shutting down at several points. However, ADMS demonstrates superior thermal management during the initial phase. CPU temperature rises more gradually and remains below the throttling threshold throughout the 10-minute window, maintaining a stable frequency range around 2.8-3 GHz. For the GPU, ADMS initially sustains high frequency (around 850-900 MHz), with a controlled gradual reduction as temperature rises, avoiding the extreme fluctuations observed with TFLite.

As shown in Table VII, Band reached thermal throttling at 9.7 minutes, while ADMS operates without throttling until 13.9 minutes. This balanced thermal management enables ADMS to sustain higher average processor frequencies even under extended workloads, resulting in more stable performance and improved user experience. The superior thermal management is attributed to ADMS's intelligent load distribution across heterogeneous processors and its ability to dynamically adjust scheduling based on real-time temperature monitoring. These experimental results confirm that ADMS not only delivers superior performance in terms of latency and throughput but also achieves improved energy efficiency and system robustness under stress conditions, making it a comprehensive solution for multi-DNN inference on mobile devices.

I. Overhead Analysis of ADMS

Although ADMS delivers significant performance improvements, a rigorous evaluation requires quantifying the overhead of its runtime mechanisms. We analyze three primary sources: processor state monitoring, scheduling logic, and memory footprint.

- *Monitoring Overhead:* The hardware monitor periodically collects system status. As detailed in Section III-C, each data retrieval cycle requires approximately 10 ms. With a refresh interval of 560 ms, this imposes a CPU duty cycle of only about 1.8% (10 ms / 560 ms).
- *Scheduling Overhead:* The multi-factor scheduler forms the core of our runtime logic. Our benchmarks show that a single scheduling decision, which calculates priority scores for queued tasks, takes less than 1 ms on average.
- *Memory Footprint:* ADMS introduces a modest memory overhead for storing performance profiles, subgraph configurations, and task queues. On our test platforms, this amounted to an additional 1–2 MB of RAM over the vanilla TFLite framework.

Crucially, the most computationally intensive task, subgraph partitioning, is performed entirely offline and thus incurs zero runtime overhead. In conclusion, the overhead introduced by the additional runtime mechanisms of ADMS is negligible. The low-latency scheduling and monitoring, combined with a small memory footprint, confirm that the substantial gains in inference performance (up to 4.04×) and energy efficiency (up to 24.2%) far outweigh the framework's costs. This analysis confirms that our design choices, which favor a lightweight heuristic scheduler, are well-justified for resource-constrained mobile environments. The negligible overhead demonstrates that ADMS does not become a bottleneck, but rather an efficient enabler of high performance, confirming its robustness in complex scenarios.

V. Related Work

A. DNN Optimization and Acceleration Techniques

Researchers have developed various techniques to optimize DNN performance on resource-constrained mobile devices. Han et al. [43] introduced deep compression techniques, including pruning, quantization, and Huffman coding, to reduce model size while maintaining accuracy. Wei et al. [44] proposed neural network branching for parallel inference, enabling better adaptation to hardware heterogeneity. Although these model optimization approaches significantly improved inference efficiency, they lacked dynamic coordination capabilities for multi-model scenarios.

Another major research direction focuses on single-processor optimization. The Pipe-it framework [45] improved DNN inference throughput on ARM big.LITTLE processors through layer partitioning and pipelining. AsyMo [14] utilized asymmetric heterogeneous CPU cores with task allocation and block partitioning strategies. DeepMon [46] enhanced vision applications on mobile GPUs via device-specific optimizations. While these works demonstrated notable performance gains on individual

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

processors, they did not address the challenges of holistic resource management across heterogeneous computing units.

B. Multi-Processor and Multi-DNN Coordination

Coordinated execution across multiple processors has emerged as a promising approach to enhance mobile DNN performance. Tan et al. [27] proposed thermal-aware scheduling that intelligently distributes workloads between GPUs and NPUs to prevent overheating. DART [26] optimized real-time inference of multiple DNN models through pipelining and data parallelism. While these approaches laid important foundations for thermal management and multi-model coordination, they were limited in processor coverage.

Cross-processor collaboration has been further explored in several systems. LaLaRAND [25] utilized hierarchical scheduling to allocate DL tasks between CPUs and GPUs. Xu et al. [47] and Jia et al. [24] have attempted to run multi-DNN inference on heterogeneous processors but typically assume specific task requirements and provide solutions tailored to limited scenarios. Wang et al. [29] optimized collaborative CPU-GPU computing under thermal constraints, while Sparse-DySta [28] enhanced multi-DNN workload performance through sparse data processing and hybrid scheduling. Although these approaches achieved notable gains in specific scenarios, most focused on limited processor combinations rather than fully leveraging all available computing resources.

Despite these advancements, existing frameworks remain constrained by their reliance on specific hardware configurations or partial processor subsets. This work addresses these limitations by providing a unified system that dynamically balances workloads across all heterogeneous processors. It utilizes optimized-granularity subgraph partitioning and real-time scheduling based on comprehensive processor status monitoring. This design enables efficient handling of multiple concurrent DNN inferences without requiring extensive model modifications.

VI. Conclusions and Future Work

In this paper, we proposed ADMS, a framework that enables efficient concurrent inference of multiple DNNs on heterogeneous mobile processors through optimized subgraph partitioning and dynamic scheduling. Experimental results on Huawei P20 and Redmi K50 Pro demonstrate that ADMS reduces inference latency by up to 4.04× compared to TFLite and improves energy efficiency by 24.2% compared to Band. Although ADMS is presented as a general framework, its current implementation has a key limitation: the partitioning algorithm is specifically designed for models with a DAG structure and does not yet support architectures with cyclic dependencies, such as Recurring Neural Networks (RNNs). In the future, we aim to adapt the framework to accommodate recurrent models like RNNs, develop real-time adaptive window size optimization to adjust partitioning granularity, integrate predictive models for proactive scheduling, and explore dynamic model quantization techniques tailored to specific hardware.

References

[1] M. Liu, X. Ding, and W. Du, "Continuous, real-time object detection on mobile devices without offloading," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 976–986.

[2] Q. Zhang et al., "Intelligence inference on IoT devices," in *Proc. Learn. Techn. Internet Things*, 2023, pp. 171–195.

[3] J. Yi, S. Choi, and Y. Lee, "Eagleeye: Wearable camera-based person identification in crowded urban spaces," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, 2020, pp. 1–14.

[4] J. Park, Y. Boo, I. Choi, S. Shin, and W. Sung, "Fully neural network based speech recognition on mobile and embedded devices," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 10642–10653.

[5] J. You, W. Liu, and J. Lee, "A DNN-based semantic segmentation for detecting weed and crop," *Comput. Electron. Agriculture*, vol. 178, 2020, Art. no. 105750.

[6] A. Albanese, M. Nardello, and D. Brunelli, "Automated pest detection with DNN on the edge for precision agriculture," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 3, pp. 458–467, Sep. 2021.

[7] W. B. Demilie, "Plant disease detection and classification techniques: A comparative study of the performances," *J. Big Data*, vol. 11, no. 1, 2024, Art. no. 5.

[8] S. A. Bhat, I. Hussain, and N.-F. Huang, "Soil suitability classification for crop selection in precision agriculture using GBRT-based hybrid DNN surrogate models," *Ecological Inform.*, vol. 75, 2023, Art. no. 102109.

[9] Q. Zhang et al., "A comprehensive deep learning library benchmark and optimal library selection," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5069–5082, May 2024.

[10] Google, "Tensorflow lite: Deploy machine learning models on mobile and IoT devices," 2019. [Online]. Available: https://www.tensorflow.org/lite/

[11] P. mobile, "Official website of pytorch mobile," 2019. [Online]. Available: https://pytorch.org/mobile/home/

[12] "CoreML: Integrate machine learning models into your app," 2022. [Online]. Available: https://developer.apple.com/documentation/coreml

[13] C.-J. Wu et al., "Machine learning at facebook: Understanding inference at the edge," in *Proc. IEEE Int. Symp. High Perform. Comput. Architecture*, 2019, pp. 331–344.

[14] M. Wang, S. Ding, T. Cao, Y. Liu, and F. Xu, "ASYMO: Scalable and efficient deep-learning inference on asymmetric mobile cpus," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 215–228.

[15] B. Gomatheeshwari and J. Selvakumar, "Appropriate allocation of workloads on performance asymmetric multicore architectures via deep learning algorithms," *Microprocessors Microsyst.*, vol. 73, 2020, Art. no. 102996.

[16] J. S. Jeong et al., "Band: Coordinated multi-DNN inference on heterogeneous mobile processors," in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2022, pp. 235–247.

[17] Q. Zhang et al., "A comprehensive benchmark of deep learning libraries on mobile devices," in *Proc. ACM Web Conf.*, 2022, pp. 3298–3307.

[18] S. S. Girija, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *Softw. Available Tensorflow. Org*, vol. 39, no. 9, 2016.

[19] Google, "Edge TPU," 2020. [Online]. Available: https://cloud.google.com/edge-tpu/

[20] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[21] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 10781–10790, 2020.

[22] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.

[23] Google, "Nnapi," 2017. [Online]. Available: https://developer.android.com/ndk/guides/neuralnetworks,

[24] F. Jia et al., "CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices," in *Proc. 20th ACM Int. Conf. Mobile Syst., Appl., Serv.*, 2022, pp. 209–221.

[25] W. Kang, K. Lee, J. Lee, I. Shin, and H. S. Chwa, "LaLaRAND: Flexible layer-by-layer CPU/GPU scheduling for real-time DNN tasks," in *Proc. IEEE Real-Time Syst. Symp.*, 2021, pp. 329–341.

[26] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 392–405.

[27] T. Tan and G. Cao, "Thermal-aware scheduling for deep learning on mobile devices with NPU," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 10706–10719, Dec. 2024.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.

[28] H. Fan, S. I. Venieris, A. Kouris, and N. Lane, "Sparse-dysta: Sparsity-aware dynamic and static scheduling for sparse multi-DNN workloads," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2023, pp. 353–366.
[29] S. Wang, G. Ananthanarayanan, and T. Mitra, "Optic: Optimizing collaborative CPU–GPU computing on mobile devices with thermal constraints," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 38, no. 3, pp. 393–406, Mar. 2019.
[30] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.
[31] B. A. Shirazi, A. R. Hurson, and K. M. Kavi, *Scheduling and Load Balancing in Parallel and Distributed Systems*. Hoboken, NJ, USA: Wiley, 1995.
[32] J. A. Stankovic, M. Spuri, K. Ramamritham, and G. C. Buttazzo, *Deadline Scheduling for Real-Time Systems: EDF and Related Algorithms*. New York, NY, USA: Springer, 1998.
[33] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 19, no. 4, pp. 1–12, 1989.
[34] J. H. Anderson and A. Srinivasan, "Early-release fair scheduling," in *Proc. 12th Euromicro Conf. Real-Time Syst.*, 2000, pp. 65–72.
[35] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
[36] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.
[37] A. Farhadi and J. Redmon, "YOLOv3: An incremental improvement," *Comput. Vis. Pattern Recognit.*, Springer, Berlin/Heidelberg, Germany, vol. 1804, pp. 1–6, 2018.
[38] X. Zhou et al., "East: An efficient and accurate scene text detector," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5551–5560.
[39] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, "Retinaface: Single-shot multi-level face localisation in the wild," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 5203–5212.
[40] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4690–4699.
[41] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
[42] B. Sedlak, V. C. Pujol, P. K. Donta, and S. Dustdar, "Markov blanket composition of SLOs," in *Proc. IEEE Int. Conf. Edge Comput. Commun.*, 2024, pp. 128–138.
[43] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantizat ion and huffman coding," 2015, *arXiv:1510.00149*.
[44] J. Wei et al., "NN-stretch: Automatic neural network branching for parallel inference on heterogeneous multi-processors," in *Proc. 21st Annu. Int. Conf. Mobile Syst., Appl. Serv.*, 2023, pp. 70–83.
[45] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, "High-throughput CNN inference on embedded ARM big.LITTLE multicore processors," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2254–2267, Oct. 2020.
[46] L. N. Huynh, Y. Lee, and R. K. Balan, "Deepmon: Mobile GPU-based deep learning framework for continuous vision applications," in *Proc. 15th Annu. Int. Conf. Mobile Syst., Appl., Serv.*, 2017, pp. 82–95.
[47] D. Xu et al., "Niagara: Scheduling DNN inference services on heterogeneous edge processors," in *Proc. Int. Conf. Service-Oriented Comput.*, 2023, pp. 67–85.



Yunquan Gao received the PhD degree from the Beijing University of Posts and Telecommunications. He is currently a lecturer with the School of Computer Science and Technology, Anhui Engineering Research Center for Intelligent Applications and Security of Industrial Internet, Anhui University of Technology, Ma'anshan, China. He has authored or coauthored papers in MSWiM and IoTJ. His research interests include IoT networks, wireless communication, mobile edge computing, on-device AI inference.



Zhiguo Zhang received the master's degree from the School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, China, in 2025. He is currently an engineer with China Telecom Corporation Ltd., Suzhou Branch, Suzhou, China. His research interests include deep learning edge inference, improving the efficiency of deep learning models for edge device deployment, with an emphasis on real-time processing, and resource-constrained environments.



Praveen Kumar Donta (Senior Member, IEEE) received the bachelor's and master's in technology (with distinction) degrees from JNTUA, Ananthapur, in 2014 and 2012, respectively, and the PhD degree from IIT (ISM), Dhanbad, in 2021. He was a visiting PhD fellow with Mobile&Cloud Lab, University of Tartu, Estonia, from 2019 to 2020. He was with Distributed Systems Group, TU Wien, as a postdoctoral researcher from 2021 to 2024. He is currently an associate professor (docent) with the Department of Computer and Systems Sciences, Stockholm University, Sweden. His research focuses on learning-driven distributed computing continuum systems. He is also the Editorial board member of *IEEE Internet of Things Journal*, *Computing* Springer, ETT Wiley, and *Scientific Reports* Journals.

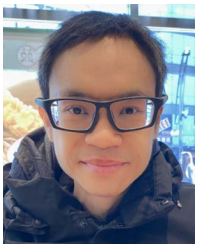


Chinmaya Kumar Dehury (Member, IEEE) received the PhD degree with the Department of Computer Science and Information Engineering, Chang Gung University, Taiwan. He was an assistant professor with the Institute of Computer Science, University of Tartu, Estonia. He is currently an assistant professor with Computer Science Department, IISER Berhampur, India. His research interests include scheduling, resource management and fault tolerance problems of cloud and fog computing, the application of artificial intelligence in cloud management, edge intelligence, Internet of Things, and data management frameworks. His research results are authored or coauthored by top-tier journals and transactions such as *IEEE Transactions on Cloud Computing*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Parallel and Distributed Systems*, and FGCS. He is a member of ACM India. He is also a PC member of several conferences and reviewer to several journals and conferences, such as *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Neural Networks and Learning Systems*, and Wiley *Software: Practice and Experience*



Xiujun Wang received the BS degree in computer science and technology from Anhui Normal University, and the PhD degree in computer software and theory from the University of Science and Technology of China. He is currently an associate professor with the School of Computer Science and Technology, Anhui University of Technology. His research interests include data stream processing, randomized algorithms, and Internet of Things.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.



Dusit Niyato (Fellow, IEEE) received the BEng degree from the King Mongkuts Institute of Technology Ladkrabang (KMITL), Thailand, and the PhD degree in electrical and computer engineering from the University of Manitoba, Canada. He is currently a professor with the College of Computing and Data Science, Nanyang Technological University, Singapore. His research interests include mobile generative AI, edge intelligence, quantum computing and networking, and incentive mechanism design.



Qiyang Zhang (Member, IEEE) received the PhD degree in computer science from the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. He was a visiting student with Distributed Systems Group, TU Wien, from 2022 to 2023. He is currently a postdoctoral researcher with Peking University. He has authored or coauthored papers in WWW, INFOCOM, *IEEE Transactions on Mobile Computing*, and *IEEE Transactions on Services Computing*. His research interests include mobile edge computing and edge intelligence.

Authorized licensed use limited to: Duale Hochschule Mannheim (FH). Downloaded on September 17,2026 at 09:41:06 UTC from IEEE Xplore. Restrictions apply.